3. GÜN TESLİMİ

Depo, Branch, Görev Panosu ve API Sözleşmesi

Team Task Manager · Ekip çalışma ve entegrasyon standardı

DEPO MODELİ	BRANCH AKIŞI	API STANDARDI
Monorepo	**main + develop**	**OpenAPI 3.0.3**

Gün sonu çıktısı: Git deposuna hazır dosya düzeni, branch ve PR politikası, GitHub issue şablonları, 20 maddelik görev panosu ve kimlik, kullanıcı, proje, görev, yorum, dosya, bildirim ve raporlama uç noktalarını kapsayan API sözleşmesi taslağı oluşturuldu.

Üretilen Artefaktlar

GIT
Çalışma Standardı
.gitignore, CONTRIBUTING, branch adları ve commit standardı.

GITHUB
İş Akışı
Feature/bug issue formları, PR şablonu ve görev panosu.

API
Sözleşme
OpenAPI YAML, JWT, ortak cevaplar, sayfalama ve endpointler.

Uzak depo durumu: GitHub CLI bu bilgisayarda kurulu değil; hesap/organizasyon ve görünürlük seçimi de belirtilmedi. Bu nedenle uzak repo oluşturulmadı, yerel yapı push işlemine hazırlandı.

1. GitHub Depo Kararı

30 günlük staj projesinde backend, web, mobil, dokümantasyon ve ortak sözleşmenin tek depoda tutulması önerilmiştir. Bu yaklaşım küçük ekipte çapraz istemci değişikliklerini tek pull request ve tek görev panosuyla izlemeyi kolaylaştırır.

```
team-task-manager/
├── src/backend/
├── client-web/team-task-manager-react/
├── client-mobile/<selected-tech>/
├── docs/
├── .github/
├── TeamTaskManager.sln
└── README.md
```

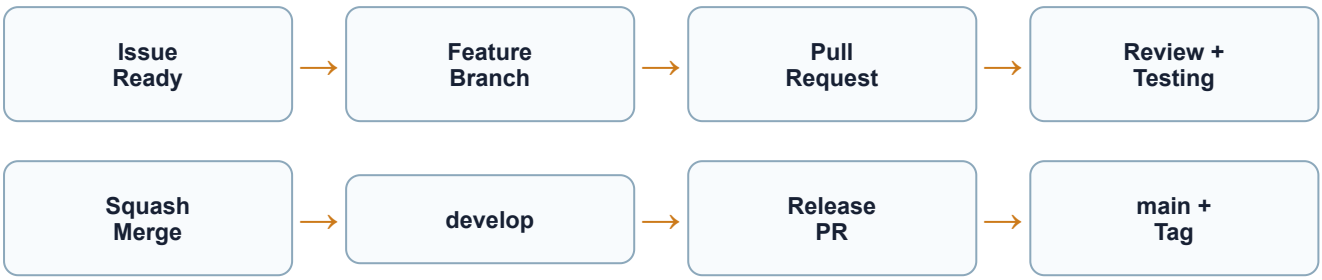
Koruma ve Sürümleme

- **main:** yalnızca yayınlanabilir/demo edilebilir kod; doğrudan push kapalı.
- **develop:** günlük entegrasyon; feature PR'larının hedefi.
- En az bir inceleme ve başarılı build/test kontrolü zorunlu.
- `develop` PR'larında squash merge; sürüm PR'larında merge commit.
- Semantik sürüm etiketleri: `v0.1.0`, `v0.2.0`, `v1.0.0`.

Gizli Bilgi ve Üretilmiş Dosyalar

Depoya alınmaz	Depoya alınır
bin/obj, node_modules, dist, uploads	Kaynak kod ve migration dosyaları
appsettings.Development.json, .env.*	.env.example ve güvenli ayar şablonları
IDE kullanıcı ayarları ve test çıktıları	Solution, doküman, test kodu ve OpenAPI

2. Branch ve Pull Request Akışı



Branch adlandırma

Tür	Kalıp	Örnek
Özellik	feature/<issue>-<ad>	feature/42-task-create
Hata	bugfix/<issue>-<ad>	bugfix/57-inactive-login
Acil düzeltme	hotfix/<issue>-<ad>	hotfix/61-token-expiry
Doküman	docs/<issue>-<ad>	docs/15-api-contract
Test	test/<issue>-<ad>	test/70-task-service

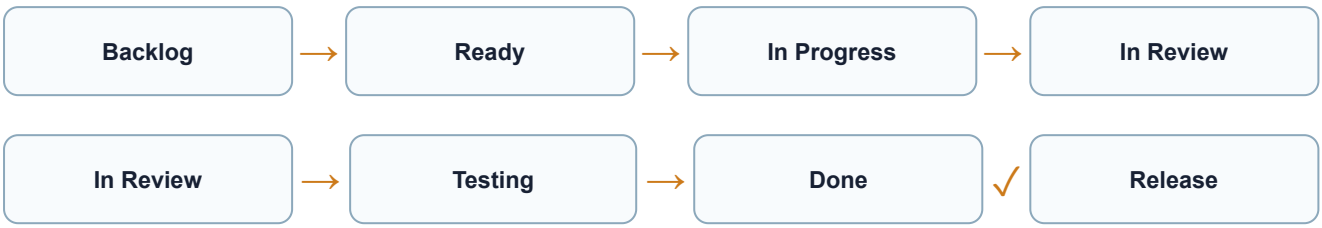
Commit standardı

```
feat(api): add task creation endpoint
fix(auth): reject inactive users
docs(api): update project contract
test(tasks): cover invalid status transition
```

PR tamamlanma koşulları

- Issue ve kabul kriteri bağlantısı mevcut.
- Derleme ve ilgili testler başarılı.
- API değiştiyse OpenAPI taslağı güncel.
- Gizli bilgi ve üretilmiş dosya eklenmemiş.

3. Görev Panosu Taslağı



Özel alanlar

Alan	Değerler
Öncelik	P0 · P1 · P2
Alan	Backend · Web · Mobile · Database · Test · Documentation
Sprint	Week 1–6
Tahmin	1 · 2 · 3 · 5 · 8 puan

Başlangıç backlog özeti

Grup	İş sayısı	Planlanan gün
Solution, entity, DbContext ve migration	5	4–6
Auth, kullanıcı, proje ve üyelik	4	7–10
Görev, yorum, dosya ve bildirim	5	11–15
Kalite, React, mobil ve teslim	6	16–30

WIP kuralı: Bir geliştiricinin aynı anda en fazla bir ana görevi In Progress durumunda olmalıdır. Bloke işler issue yorumunda neden ve ihtiyaç duyulan aksiyonla belirtilir.

4. API Tasarım Kararları

Base URL
`/api` kökü ve çoğul kaynak adları.

Kimlik
JWT Bearer; yalnız login anonim.

Format
JSON, camelCase, ISO-8601 tarihler.

Standart cevap zarfı

```
{
  "success": true,
  "message": "Görev başarıyla oluşturuldu.",
  "data": { ... },
  "errors": []
}
```

HTTP durum kodları

Kod	Kullanım
200 / 201 / 204	Başarılı okuma / oluşturma / gövdesiz güncelleme
400	Giriş doğrulama veya geçersiz istek
401 / 403	Kimlik yok/geçersiz veya yetki yok
404	Kayıt yok ya da güvenlik gereği görünmez
409	İş kuralı/benzersizlik/durum geçişi çakışması
413	Dosya boyutu sınırını aştı

Sayfalama ve sıralama

`pageNumber=1&pageSize=20&sort=dueDate:asc`; maksimum sayfa boyutu 100. Liste cevabı `items`, `pageNumber`, `pageSize`, `totalCount`, `totalPages` alanlarını içerir.

5. Endpoint Grupları

Grup	Temel uç noktalar	Yetki
Auth	POST login, GET me, POST change-password	Login anonim; diğerleri giriş yapan
Users	GET/POST users, GET/PUT user, PATCH status	Admin; detayda yönetici politikası
Projects	GET/POST projects, GET/PUT detail, PATCH status	Rol + proje kaynağı
Members	GET/POST members, DELETE member	Proje yöneticisi
Tasks	GET tasks/my/detail, POST/PUT, PATCH status, DELETE	Üyelik + atama/yönetici
Comments	GET/POST comments, PUT/DELETE comment	Proje üyesi + sahiplik
Attachments	POST upload, GET download, DELETE	Üyelik + yükleyen/yönetici
Notifications	GET list, PATCH read/read-all	Bildirim sahibi
Dashboard	GET summary, project-progress, workload	Rol bazlı kapsam

6. Taslak Şemalar

- **LoginRequest / LoginResponse:** e-posta, parola, access token, süre ve kullanıcı özeti.
- **ProjectCreateRequest:** ad, açıklama, başlangıç ve hedef bitiş tarihi.
- **TaskCreateRequest:** proje, başlık, açıklama, atanan kişi, öncelik ve tarihler.
- **TaskStatusUpdateRequest:** hedef durum; geçiş business katmanında doğrulanır.
- **Enumlar:** ProjectStatus, TaskStatus ve TaskPriority string olarak sözleşmede gösterilir.

> **Sözleşme kuralı:** Backend, React ve mobil ekip aynı `docs/api-contract-draft.yaml` dosyasını referans alır. Uygulama Swagger çıktısıyla ayrışırsa sözleşme aynı PR içinde güncellenir.

7. Oluşturulan Dosyalar

Dosya	Amaç
.gitignore	.NET, IDE, istemci, secret ve çıktı dışlamaları
CONTRIBUTING.md	Branch, commit ve PR standardı
.github/ISSUE_TEMPLATE/*	Özellik ve hata issue formları
.github/PULL_REQUEST_TEMPLATE.md	PR açıklaması ve kontrol listesi
docs/03-branch-stratejisi.md	Kalıcı/kısa ömürlü branch politikası
docs/03-gorev-panosu.md	Panonun sütun, alan, backlog ve DoD taslağı
docs/api-contract-draft.yaml	OpenAPI 3.0.3 API sözleşmesi

8. Tamamlanma Kontrolü

- ✓ Monorepo kararı verildi.
- ✓ Branch ve merge politikası yazıldı.
- ✓ Commit standardı belirlendi.
- ✓ Issue ve PR şablonları oluşturuldu.
- ✓ 20 maddelik backlog hazırlandı.
- ✓ Definition of Done tanımlandı.
- ✓ API endpointleri taslaklandı.
- ✓ Ortak cevap/hata yapısı belirlendi.

9. GitHub'a Aktarma Adımları

1. GitHub'da private `team-task-manager` deposu oluştur.
2. Yerel `main` dalını `origin` uzak deposuna bağla ve push et.
3. `develop` dalını oluşturup varsayılan geliştirme dalı olarak kullan.
4. Branch protection kurallarını etkinleştir.
5. GitHub Project panosunu oluşturup 20 backlog maddesini issue olarak içe aktar.

> **Sonuç:** 3. gün planlama çıktısı tamamlandı. Proje artık 4. gün N katmanlı .NET solution kurulumuna başlayabilecek iş akışı ve API sınırlarına sahiptir.